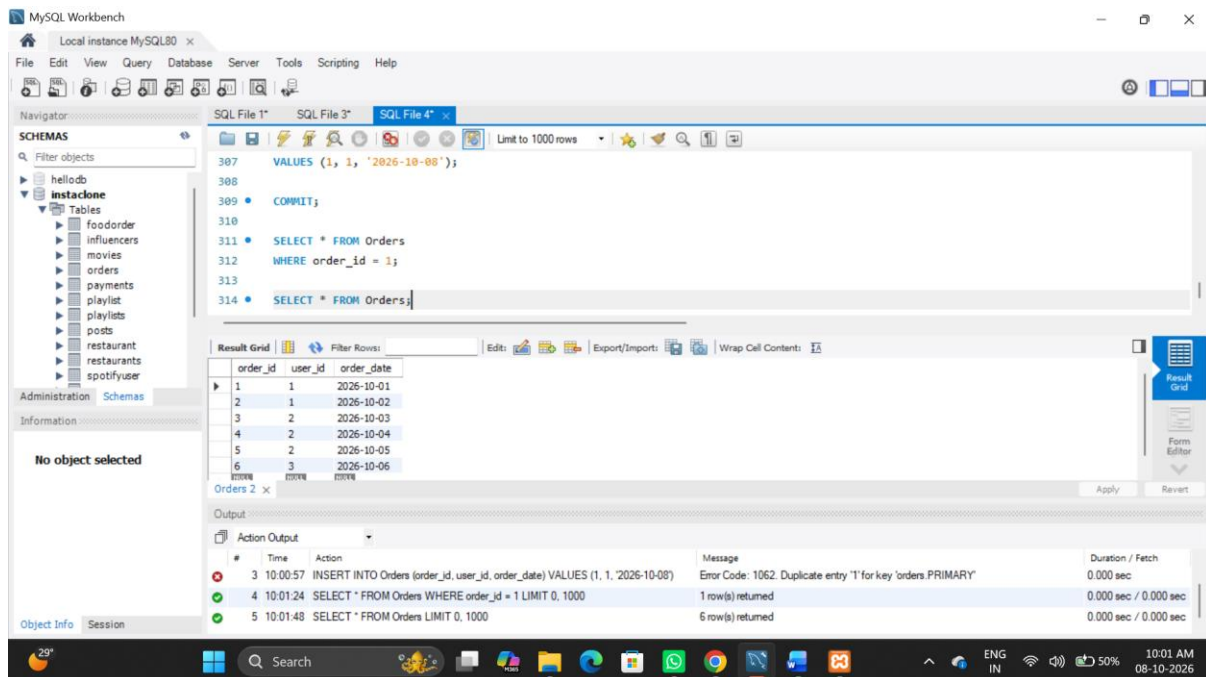


Session – 9

Q1. Create a SQL script to insert a new order into an Orders table, then use COMMIT to save the transaction and verify that the new order persists after reconnecting to the database.



The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' tree with 'instacore' selected. The main editor shows a SQL script with the following lines:

```
307 VALUES (1, 1, '2026-10-06');
308
309 COMMIT;
310
311 SELECT * FROM Orders
312 WHERE order_id = 1;
313
314 SELECT * FROM Orders;
```

The 'Result Grid' shows the output of the queries:

order_id	user_id	order_date
1	1	2026-10-01
2	1	2026-10-02
3	2	2026-10-03
4	2	2026-10-04
5	2	2026-10-05
6	3	2026-10-06

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
3	10:00:57	INSERT INTO Orders (order_id, user_id, order_date) VALUES (1, 1, '2026-10-06')	Error Code: 1062. Duplicate entry '1' for key 'orders PRIMARY'	0.000 sec
4	10:01:24	SELECT * FROM Orders WHERE order_id = 1 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
5	10:01:48	SELECT * FROM Orders LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

Q2. Simulate a Zomato-style food order process: insert two new items into an OrderItems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain in the table after rollback.

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following script:

```
326 (1, 1, 'Pizza', 2),
327 (2, 1, 'Burger', 1);
328
329 SELECT * FROM OrderItems;
330
331 ROLLBACK;
332
333 SELECT * FROM OrderItems;
```

The Result Grid shows the output of the queries:

item_id	order_id	item_name	quantity
1	1	Pizza	2
2	1	Burger	1

The Action Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
9	10:05:57	SELECT * FROM OrderItems LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
10	10:06:05	ROLLBACK	0 row(s) affected	0.000 sec
11	10:06:10	SELECT * FROM OrderItems LIMIT 0, 1000	2 row(s) returned	0.016 sec / 0.000 sec

Q3. In a Flipkart-like shopping cart scenario, use SAVEPOINT to mark a point after adding a product to the Cart table, add another product, then use ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following script:

```
345 • SAVEPOINT product_added;
346
347 • INSERT INTO Cart (cart_id, product_name, quantity)
348   VALUES (2, 'House', 2);
349
350 • ROLLBACK TO SAVEPOINT product_added;
351
352 • SELECT * FROM Cart;
```

The Result Grid shows the output of the final SELECT statement:

cart_id	product_name	quantity
1	Laptop	1

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
13	10:12:47	INSERT INTO Cart (cart_id, product_name, quantity) VALUES (1, 'Laptop', 1)	1 row(s) affected	0.000 sec
14	10:13:00	SAVEPOINT product_added	0 row(s) affected	0.000 sec
15	10:13:17	SELECT * FROM Cart LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Q4. Write a trigger on a Wallet table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to how Paytm updates wallet balance after a payment.

The screenshot displays the MySQL Workbench interface with a local instance of MySQL80. The left sidebar shows a tree of schemas, including 'wallet' and 'wallettransaction'. The main editor window contains the following SQL code:

```
376 DELIMITER ;
377
378 • INSERT INTO Wallet VALUES (1, 5000);
379
380 • INSERT INTO WalletTransaction
381   VALUES (1, 1, 1200);
382
383 • SELECT * FROM Wallet;
```

The 'Result Grid' shows the output of the queries:

	user_id	balance
1	1	3800.00

The 'Output' tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
20	10:17:01	INSERT INTO Wallet VALUES (1, 5000)	1 row(s) affected	0.031 sec
21	10:17:05	INSERT INTO WalletTransaction VALUES (1, 1, 1200)	1 row(s) affected	0.016 sec
22	10:17:09	SELECT * FROM Wallet LIMIT 0.1000	1 row(s) returned	0.000 sec / 0.000 sec

Q5. Use ChatGPT to generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining how you fixed the error.

Hint: Ask ChatGPT for a buggy trigger example and a test case that exposes the bug.

